

Ensemble Methods

Oxford Spring School in Advanced Research Methods

Dr Thomas Robinson, LSE

Day 5 of 5

Recap

Over past three sessions:

- ▶ Explored three different broad categories of ML estimation:
 1. Extensions to OLS
 2. Tree-based methods
 3. Neural networks

Each attempt to make good out-of-sample in different ways, but share some foundational concepts:

- ▶ Bias-variance trade off
- ▶ Regularisation
- ▶ Hyperparameter tuning

How do we know which model to use and when?

- ▶ What if different models yield different predictions?

Today's session

Introduce the concept of ensemble learning:

- ▶ Combine multiple estimators to *improve* the overall predictive performance
- ▶ Different estimators might capture different components of covariance
- ▶ Can we combine the methods into a “super-learner”?

Rest of the session:

1. Basic ensemble methods
2. Superlearner theory
3. Causal inference and ML

A strong modern baseline library

In practice, a useful ensemble library often includes:

- ▶ A simple benchmark such as mean prediction / majority class / GLM
- ▶ A regularised linear model
- ▶ At least one tree ensemble
- ▶ A more flexible learner such as BART or a neural network when justified

The goal is diversity in predictive bias, not just more versions of the same model.

Returning to the running case study

By the end of the week, the case study has become:

- ▶ one outcome
- ▶ several different learners
- ▶ different forms of predictive error
- ▶ a decision about whether extra complexity is actually buying us anything

How should we score a predictive model?

Accuracy is useful, but often incomplete.

Other important metrics include:

- ▶ Log loss
 - ▶ rewards well-calibrated probabilities and punishes overconfidence
- ▶ Brier score
 - ▶ squared error for predicted probabilities
- ▶ Precision and recall
 - ▶ useful when one class is rare or costly to miss
- ▶ ROC / PR trade-offs
 - ▶ useful when the threshold itself is part of the decision problem

Calibration matters

Two models can have the same accuracy but very different probabilities.

If a model predicts:

- ▶ 0.90, it should be right about 90% of the time
- ▶ 0.20, it should be right about 20% of the time

Calibration matters when predicted probabilities are used for:

- ▶ targeting interventions
- ▶ ranking cases for audit
- ▶ downstream decision-making
- ▶ uncertainty communication in research

Grouped performance is substantive, not optional

Always ask:

- ▶ Which observations are hardest to predict?
- ▶ Does performance differ by gender, race, class, region, or time?
- ▶ Is the model learning real structure or exploiting shortcuts?

Average performance can hide politically and scientifically important failures.

ML as learning

To go back to the fundamentals of ML:

- ▶ Different models represent different ways to *learn* the relationship between $\mathbf{X}$ and $\mathbf{y}$
 - ▶ By projecting points onto a lower dimensional surface (OLS)
 - ▶ By subsetting data into ever more homogenous groups (trees)
 - ▶ By approximating complex functional forms (neural nets?)

We can think about two levels of optimisation:

- ▶ $\arg \min_{\lambda, \theta} |f(\mathbf{X}) - \hat{g}_{\lambda, \theta}(\mathbf{X})|$
 - ▶ Solved algorithmically and via tools like cross-validation
- ▶ $\arg \min_g |f(\mathbf{X}) - \hat{g}_{\lambda, \theta}(\mathbf{X})|$
 - ▶ Need to estimate all $g \in \mathcal{G}$, and perform tuning within each g

Generic meta-prediction problem

How can we optimise our choice of “learner” in order to best approximate the true data generating process?

- ▶ Multiple strategies to choose from
- ▶ Multiple specific estimators within strategies

Naive approach:

- ▶ Run multiple models, choose the one with lowest training error
- ▶ *Why might this not be the best strategy?*

Combining models

Choosing a *single* best model may be suboptimal when:

- ▶ Different estimating strategies better estimate different parts of the outcome

The alternative is to combine models:

- ▶ We can average models together
- ▶ We can adapt models sequentially
- ▶ We can “weight” and sum models

Do any of these sound familiar?

Overarching intuition is to leverage the different capacities of an **ensemble** of models in order to improve prediction performance

Ensemble methods

Three basic approaches to combining ML models

Bagging

- ▶ “Bootstrap aggregation”, random samples with replacement as training data
- ▶ Typically used to reduce variance

Boosting

- ▶ Sequential weak-learner models, data iteratively weighted by misclassification
- ▶ Typically used to reduce bias

Stacking

- ▶ Run different estimation strategies on the *same* training data, then combine estimates
- ▶ Removes model selection issues and improves predictive performance

Bagging

We've already briefly covered bagging – it helps convert CART models into a random forest

- ▶ Bagging = “**Bootstrap aggregation**”
- ▶ We train B *independent* models of the same type
 - ▶ If $\mathbf{X}$ were the same across all B models, and assuming a deterministic estimator, then there would be no variance in the predictions
 - ▶ Therefore we induce variation by bootstrapping the data
 - ▶ Let n be the number of rows in $\mathbf{X}$
 - ▶ Randomly sample n rows of the data *with replacement*
 - ▶ Yields B datasets of the same size as $\mathbf{X}$ (with the same variables)
 - ▶ Rows of $\mathbf{X}$ can repeat within any given sample

Bagging

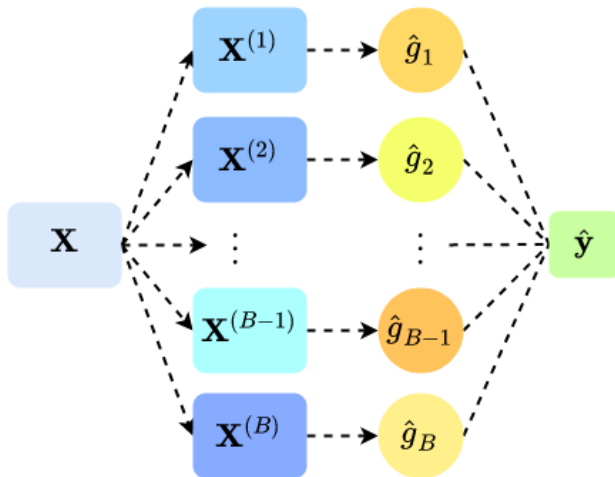
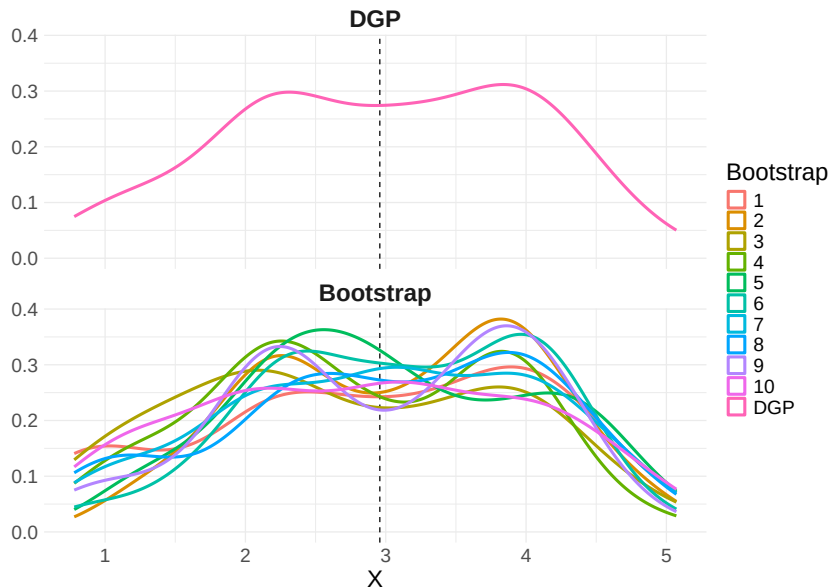


Figure 1: Schematic of a bagged ensemble

Visualising the bootstrap



Bagging details

If $\mathbf{X}$ is the training data:

- ▶ Let $\mathcal{B} = \{\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(B)}\}$ be a set of randomly resampled datasets, each the length of $\mathbf{X}$
- ▶ Let g be an estimating function
- ▶ $\mathbf{X}'$ be the test data

Then:

- ▶ For each dataset $\mathbf{X}^{(b)} \in \mathcal{B}$, estimate $\hat{g}_b = g(\mathbf{X}^{(b)})$
- ▶ Then, pass the test data through each separate model
 - ▶ $\hat{\mathbf{y}}_b = \hat{g}_b(\mathbf{X}')$
- ▶ $\hat{\mathbf{y}}_i = \frac{1}{B} \sum_{b=1}^B \hat{y}_{ib}$

Why is bagging useful?

Bagging is typically used with **high variance** estimators:

- ▶ I.e. the modelling strategy likely overfits the data
- ▶ Yielding predictions that are very sensitive to the input data
- ▶ By bootstrapping $\hat{X}$ we can smooth out this sensitivity
- ▶ And yield predictions that exhibit less variance

Categorical outcomes

The above strategy works very well for continuous data, but what about categories?

Two methods:

1. **Hard vote**

- ▶ Each model acts as a “voter”, choosing the class with the highest predicted probability *within* that model
- ▶ $\hat{y}_i$ gets assigned the class with the most “votes” across the B models

2. **Soft vote**

- ▶ For every class, average the predicted probabilities from the B models
- ▶ $\hat{y}_i$ gets assigned the class with the highest *average* predicted probability

Parallelisation

Particularly convenient aspect of bagging is that it can be **parallelised**:

- ▶ Since each model is independent, the estimation is entirely separate
- ▶ Thus, separate compute units can be used on each model
- ▶ Or, more realistically:
 - ▶ If we have C cores on a PC, each core can train $\frac{B}{C}$ models
 - ▶ In other words, C models can be trained at once
 - ▶ If the actual computation is not too taxing, then this can dramatically improve speed of training

Parallelisation is used across ML:

- ▶ Some calculations are very simple but there are simply thousands of them
- ▶ Can shift these away from CPU to GPU, as used in neural network training

Boosting

Bagging helps address the problem of high variance methods

- ▶ What about high *bias* methods?
- ▶ We might have modeling strategies that exhibit low variance. . .
 - ▶ E.g. (very) shallow CART trees
- ▶ . . . but are poor predictors of some observations

Boosting operates on the principle of training models sequentially

- ▶ The next model g_{m+1} attempts to address parts of $\mathbf{y}$ that g_m predicted less well
 - ▶ I.e. correcting previous mistakes
- ▶ One common approach is to boost using **adaptive boosting**
 - ▶ Popular classification ensemble method

Adaptive boosting

Under adaptive boosting, the ensemble model is a weighted sum of “weak learners”

$$S_L(\mathbf{X}) = \sum_{l=1}^L c_l \times w_l(\mathbf{X}^{(l)}),$$

where:

- ▶ w_l is l th weak learner model, for $l \in \{1, \dots, L\}$
- ▶ c_l is a coefficient corresponding to the l th model
- ▶ S_L is the final prediction function

To optimise $\mathbf{c}$ and $\mathbf{w} = \{w_1, \dots, w_L\}$, we add the models sequentially to S , *reweighting* the data after each model

The optimisation function at each iteration is defined as:

$$\arg \min_{c_l, w_l} \sum_{i=1}^N e(y_i, S_{l-1}(\mathbf{x}_i) + c_l \times w_l(\mathbf{x}_i))$$

where e is the prediction error function

Boosting (schematic)

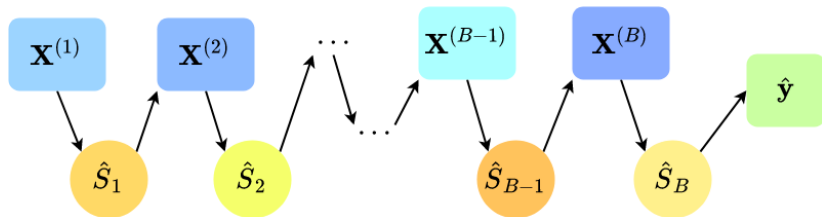


Figure 2: Schematic of a boosted ensemble

Adaptive boosting classifier algorithm

Let $\psi^{(1)} = \mathbf{1}$ (the initial weights for each observation in $\mathbf{X}$)

For l in $1 : L$

1. Find w_l that minimises the following loss

$$\epsilon_l = \frac{\sum_{i=1}^N \psi_i^{(l)} \times \mathbb{I}(y_i \neq w_l(\mathbf{x}_i))}{\sum_i \psi_i^{(l)}}$$

2. Given $\hat{w}_l$, compute its coefficient

$$c_l = \ln \frac{1 - \epsilon_l}{\epsilon_l}$$

3. Update each observation weight, such that

$$\psi_i^{(l+1)} = \psi_i^{(l)} \times e^{c_l \mathbb{I}(w_l(\mathbf{x}_i) \neq y_i)}$$

Then, $\hat{S}_L(\mathbf{X}) = \text{sign}(\sum_{l=1}^L c_l \hat{w}_l(\mathbf{X}))$

Parallelisation?

We cannot boost a model in parallel

- ▶ Each subsequent model is fit to address the deficiencies of the pre-existing model
- ▶ Thus S_l is dependent on the learner S_{l-1}

Potentially inflates training times

- ▶ To boost a model with 1000 iterations is going to take $1000 \times c$ to learn
- ▶ Offset if our low-bias individual models are simple
- ▶ Suppose we have 4 cores available
 - ▶ Complex bagged model takes $10c$ to run, and we have 1000 iterations
 - ▶ Total time = $\frac{1000}{4} \times 10c = 2500c$
 - ▶ Simple model takes $1c$ and we run 1000 models
 - ▶ Total time = $1000c$

Stacking

Stacking also works by combining models, but here allows the *type* of model to vary.

The general intuition is:

- ▶ Individual learning strategies may be “weak”
- ▶ Any one model is unlikely to be optimal
- ▶ Leverage different predictive capacities of M strategies to improve prediction

This is a meta-regression strategy:

- ▶ We assume a model H such that $\mathbf{y} = H(\mathbf{X})$
- ▶ The constituent parameters of the super-regression are the outputs from individual-learning strategies

Stacking

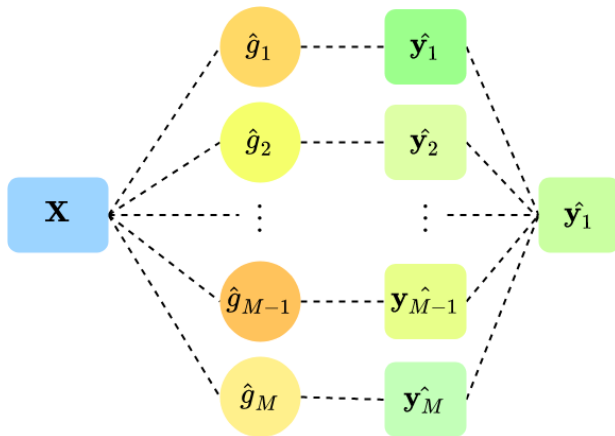


Figure 3: Schematic of a stacked ensemble

Estimation steps

Four steps to generating stacked ensemble predictor:

1. Isolate M learning strategies
2. Train M strategies using k -fold validation
3. Estimate parameters of meta-regression (the “super-learner”)
4. Predict new data using trained ensemble

1. Isolate learning strategies

Can use any modelling strategies:

- ▶ Just need to be able to recover individual $\hat{y}$ predictions
- ▶ In general, better to include more models

Model choices:

- ▶ OLS
- ▶ LASSO
- ▶ CART
- ▶ Random Forest
- ▶ BART
- ▶ Neural networks, etc.

Let $\mathcal{M}$ be the set of chosen strategies (the “library”):

- ▶ Let g_m be a learning strategy $m \in \mathcal{M}$
- ▶ $\hat{g}_m$ is the trained model on training data $\mathbf{X}$
- ▶ $\hat{y} = \hat{g}_m(\mathbf{X})$ is the predicted outcomes of the training data

2. Cross-fold validation for *each* strategy

Let $\mathcal{V}$ be a set of random folds $\{1, \dots, V\}$, where

- ▶ $\mathbf{X}^{(v)}$ is the training data from the v th fold
- ▶ $\mathbf{X}^{(-v)}$ is the training data excluding the v th fold

For $v \in \mathcal{V}$:

- ▶ Train each strategy g_m on $\mathbf{X}^{(-v)}$
- ▶ Generate predicted values for the v th fold
 - ▶ i.e. $\hat{\mathbf{y}}_m^{(v)} = \hat{g}_m(\mathbf{X}^{(v)})$

Concatenate the predictions for each m

- ▶ $\hat{\mathbf{y}}_m = \{\hat{\mathbf{y}}_m^{(1)}, \dots, \hat{\mathbf{y}}_m^{(V)}\}$

Critical detail: avoid leakage in stacking

The meta-learner should be trained on **out-of-fold** predictions, not fitted values from models trained on the same rows.

Otherwise:

- ▶ The super-learner will over-trust overfit models
- ▶ Reported performance will be too optimistic
- ▶ The final test-set comparison becomes much less credible

3. Estimate super learner weights

With M vectors of predictions for $\mathbf{y}$:

- ▶ Create a $n \times m$ matrix by stacking predictions from each model

Then we estimate a constrained model:

$$y_i = \sum_{m=1}^M w_m \hat{y}_{im} + \epsilon_i,$$

where w_m are weights, and:

- ▶ $w_m \geq 0$
- ▶ $\sum_m w_m = 1$

Denote the resultant trained super-learner model $\hat{H}$

4. Predict new data

From stages 1-3 we have:

- ▶ A library of trained models on $\mathbf{X}$
- ▶ Estimated weights for each model in $\mathcal{M}$

To make predictions on *new* test data $\mathbf{X}'$ we:

1. Pass the test data through each model in $\mathcal{M}$
 - ▶ I.e. $\hat{\mathbf{y}}'_m = \hat{g}_m(\mathbf{X}')$, where $\hat{g}_m$ has been pre-trained on $\mathbf{X}$
2. Pass the resultant predictions through $\hat{H}$
 - ▶ I.e. $\hat{\mathbf{y}}' = \hat{H}(\mathbf{X}') = \sum_{m=1}^M \hat{w}_m \hat{\mathbf{y}}'_m$

Oracle performance

This “super-learner” strategy is particularly effective because it is an **oracle learner**:

The cross-validated super-learner model “perform asymptotically exactly as well. . . as the best possible choice for the given data set among the family of weighted combinations of the estimators”

This means the super-learner really is “super”:

“Super learner will asymptotically outperform any of its competitors, even if the set of competitors is allowed to grow polynomial in sample size”

– Polley and Van der Laan 2010

Three kinds of uncertainty

It helps to keep three ideas separate:

- ▶ Parameter uncertainty
 - ▶ uncertainty about coefficients or weights
- ▶ Predictive uncertainty
 - ▶ uncertainty about $\hat{y}$ for a new observation
- ▶ Measurement uncertainty
 - ▶ uncertainty in a predicted covariate or label used downstream

These are related, but they are not the same object.

Parallelisation

Nothing precludes training the M models simultaneously:

- ▶ Modelling across strategies is parallel
- ▶ Devoting fewer cores to each method may yield slower aggregate performance if individual models themselves rely on parallelism (i.e. random forest)

The weighting model can only be estimated once all models are trained

- ▶ OLS regression should be trivially quick

Application: Heterogeneous Treatment Effects

The ATE is an *average*

Suppose we have 8 observations of an outcome, treatment assignment and two covariates:

y	d	Gender	Education
12	1	Female	High
13	1	Female	Low
5	0	Female	High
6	0	Female	Low
7	1	Male	High
8	1	Male	Low
7	0	Male	High
6	0	Male	Low

Table 1: Observed

y	d	Gender	Education
?	0	Female	High
?	0	Female	Low
?	1	Female	High
?	1	Female	Low
?	0	Male	High
?	0	Male	Low
?	1	Male	High
?	1	Male	Low

Table 2: Unobserved counterfactual

$$ATE_{\text{Observed}} = 10 - 6 = 4$$

The ATE may mask considerable heterogeneity

Conditional Average Treatment Effect

We can break down our ATE into conditional estimates:

$$\text{CATE} = \mathbb{E}[Y|d = 1, X = a] - \mathbb{E}[Y|d = 0, X = a]$$

In particular, we can think about estimating the individual level effect, i.e.

$$\text{ITE} = [Y_i|d = 1] - [Y_i|d = 0]$$

- ▶ This is not an average
- ▶ Conventionally impossible to estimate given **fundamental problem of causal inference**

Prediction problem:

- ▶ For those treated (control) units, what would they have done under control (treatment)?

Ensemble estimation strategy

1. Train a stacked ensemble estimator on the observed data ($\mathbf{X}$), using a library of methods:
 - ▶ BART
 - ▶ Random forest
 - ▶ Shallow CART
 - ▶ Neural network
 - ▶ etc.
- ▶ Yields $\hat{H}$ – our ensemble estimator
2. Predict $\hat{H}(\mathbf{X}^{(\neg d)})$
 - ▶ Counterfactual data where if $d_i = 1$, $\neg d_i = 0$
 - ▶ All other variables are identical
3. Calculate ITE

Causal guardrails

Even when using very flexible learners, causal claims still require design assumptions:

- ▶ No post-treatment variables in the feature set
- ▶ Sufficient overlap / positivity
- ▶ A defensible ignorability story or an explicit experimental design
- ▶ Careful checking that the model is not just learning treatment assignment

Estimating the CATE - generate two test matrices

- ▶ Predictions are made using two matrices
- ▶ Second matrix is the test dataset with predicted outcome
- ▶ Matrices are identical except treatment assignment is reversed in second matrix

$d_{\text{Obs.}}$	Gender	Education	$y_{i,d}$	$d_{\text{Counter.}}$	Gender	Education	$\hat{y}_{i,d}$
1	Female	High	14	0	Female	High	7
1	Female	Low	12	0	Female	Low	7
0	Female	High	4	1	Female	High	12
0	Female	Low	6	1	Female	Low	13
1	Male	High	7	0	Male	High	8
1	Male	Low	7	0	Male	Low	6
0	Male	High	8	1	Male	High	8
0	Male	Low	6	1	Male	Low	6

Estimating the CATE - rearrange matrices

- ▶ Matrices can be rearranged such that all observations in matrix 1 are $d = 1$ and *vice versa*
- ▶ Covariate information is constant across both matrices

$d_{\text{Obs.}}$	Gender	Education	$y_i^{(d=1)}$	$d_{\text{Counter.}}$	Gender	Education	$y_i^{(d=0)}$
1	Female	High	14	0	Female	High	7
1	Female	Low	12	0	Female	Low	7
1	Female	High	12	0	Female	High	4
1	Female	Low	13	0	Female	Low	6
1	Male	High	7	0	Male	High	8
1	Male	Low	7	0	Male	Low	6
1	Male	High	8	0	Male	High	8
1	Male	Low	6	0	Male	Low	6

Estimating the CATE - recover CATE

- ▶ $\text{CATE} = y_i^{(d=1)} - y_i^{(d=0)}$
- ▶ To check for treatment heterogeneity, append covariate information since this is constant across two matrices¹

$$\begin{pmatrix} \hat{y}_{i,d=1} \\ 14 \\ 12 \\ 12 \\ 13 \\ 7 \\ 7 \\ 6 \\ 7 \end{pmatrix} - \begin{pmatrix} \hat{y}_{i,d=0} \\ 7 \\ 7 \\ 4 \\ 6 \\ 8 \\ 6 \\ 8 \\ 6 \end{pmatrix} = \begin{pmatrix} \text{CATE} & \text{Gender} & \text{Education} \\ 7 & \text{Female} & \text{High} \\ 5 & \text{Female} & \text{Low} \\ 8 & \text{Female} & \text{High} \\ 7 & \text{Female} & \text{Low} \\ -1 & \text{Male} & \text{High} \\ 1 & \text{Male} & \text{Low} \\ -2 & \text{Male} & \text{High} \\ 1 & \text{Male} & \text{Low} \end{pmatrix}$$

For more information, see Grimmer, Messing, and Westwood (2017):
<https://doi.org/10.1017/pan.2017.15>

¹NB: all observations are predicted from posterior draws; red numbers indicate predictions using counterfactual treatment assignment

Final reminder: ML is not a panacea

ML: what can go wrong?

ML is really powerful:

- ▶ Individual models can be highly predictive
- ▶ Aggregate of M modeling strategies can be even better (and no worse!)
- ▶ Clear relevance to prediction problems

But we are not absolved of inference issues

- ▶ ML involves *biased* estimators
- ▶ By definition, these will not centre on *true* parameter estimate

Is this problematic?

- ▶ OLS may be no better (Gauss Markov assumptions are strong)
- ▶ But if GM does hold, then we *would* recover unbiased estimate

"I ran a LASSO model to identify the most important predictors from a high-dimensional dataset. For every unit increase in GDP, perceived levels of democracy increase by 20%!"

1. What's good about this statement?
2. What's bad about this statement?

“It’s impossible to directly observe levels of trust for historical periods. Therefore we take a library of portrait paintings from across time, and use a neural network-based facial recognition strategy to predict trustworthiness. When we regress a measure of democracy over time with this measure, we find that increases in trust lead to increases in democratic values!”

1. What's good about this statement?
2. What's bad about this statement?

Tracking historical changes in perceived trustworthiness in Western Europe using machine learning analyses of facial cues in paintings

[Lou Safra](#) , [Coralie Chevallier](#), [Julie Grèzes](#) & [Nicolas Baumard](#) 

Nature Communications **11**, Article number: 4728 (2020) | [Cite this article](#)

67k Accesses | **15** Citations | **2051** Altmetric | [Metrics](#)

16 September 2022 Editor's Note: We asked authors to respond to criticisms raised by readers and by the editors. We also contacted two of the original reviewers, and consulted informally with an additional expert in art history. In light of the reports from the original reviewers that we received on this response, we have worked with the authors on a correction of the original manuscript to address the inaccuracies in terminology, provide additional context to the study and highlight the limitations of the datasets used and of the assumptions made. We note that several of the points raised by readers after publication had been raised by reviewers and answered by the authors during the original peer-review process. We are publishing the original peer review file as an additional Supplementary file for the original manuscript, as well as the post publication exchange between authors, editors and reviewers as part of the Correction.

30 September 2020 Editor's Note: Readers are alerted that this paper is subject to criticisms that are being considered by the editors. A further editorial response will follow the resolution of these issues.

 An [Author Correction](#) to this article was published on 14 September 2022

 This article has been [updated](#)

Readings

Four suggested readings after this week:

- ▶ James et al., *An Introduction to Statistical Learning*, Ch. 8.2 (Bagging, Random Forests, Boosting)
- ▶ Van der Laan, Polley, and Hubbard (2007), “Super Learner,” *Statistical Applications in Genetics and Molecular Biology* 6(1)
- ▶ Mullainathan and Spiess (2017), “Machine Learning: An Applied Econometric Approach,” *Journal of Economic Perspectives* 31(2): 87–106
- ▶ Künzel et al. (2019), “Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning,” *PNAS* 116(10): 4156–4165

Well done!

A pleasure teaching you all!

- ▶ This is a **hard** and intensive course
 - ▶ Practice makes perfect
- ▶ Course materials will remain available online
- ▶ Please do stay in touch:

t.robinson7@lse.ac.uk